

Clasificación de tweets sobre desastres

Laboratorio 5 · Minería de textos y análisis de sentimiento

Fabian Prado · Abby Donis · Hansel López

Se parte del conjunto *Natural Language Processing with Disaster Tweets* de Kaggle, 7,613 tweets etiquetados según se refieran o no a un desastre real, y se construye un clasificador capaz de decidirlo sobre un tweet nuevo. El informe recorre la limpieza del texto, el análisis de frecuencias y n-gramas, los tres modelos comparados, la función de clasificación, el análisis de polaridad y, finalmente, la pregunta de si añadir una variable de negatividad mejora el clasificador.

Los tres resultados principales

1. El mejor modelo es la regresión logística con TF-IDF de unigramas y bigramas, F1 = 0.7581. La selección merece una aclaración: sobre la partición 80/20 con semilla 42 ganaba Naive Bayes con F1 = 0.7734, pero al promediar sobre quince particiones el orden se invierte y la regresión logística gana por 1.4 puntos, una ventaja que supera la desviación de ambos modelos. La ventaja de Naive Bayes existía sólo en esa partición.

2. Los tweets de desastre son más negativos, pero el efecto es moderado. La probabilidad de que un tweet de desastre tomado al azar sea más negativo que uno de la otra categoría es 0.6297, frente al 0.50 de una moneda. La diferencia es estadísticamente incontestable ($p \approx 10^{-85}$) y los dos léxicos empleados coinciden en la dirección, pero un tercio largo de las comparaciones va en sentido contrario.

3. La variable de negatividad no mejora el clasificador. Sobre los mismos tweets de validación arregla 30 clasificaciones y rompe 26 (McNemar, $p = 0.689$). La razón es que las palabras que aportan negatividad en este corpus —*fire*, *disaster*, *bomb*, *killed*— son el vocabulario de desastre, que el TF-IDF ya usa como predictoras con pesos aprendidos de los datos.

1. El conjunto de datos

El archivo `train.csv` trae **7,613 tweets** y cinco columnas: el identificador, una palabra clave, la ubicación declarada, el texto y la etiqueta. El reparto de clases está razonablemente equilibrado, con 4,342 tweets que no son desastre (57.03 %) y 3,271 que sí lo son (42.97 %). Ese equilibrio importa para la elección de métrica: la exactitud no es engañosa aquí como lo sería con un 95/5, pero se reporta F1 junto a ella porque la clase de interés es la minoritaria.

Las dos columnas auxiliares tienen valores ausentes en proporciones muy distintas: `keyword` falta en el 0.80 % de los tweets y `location` en el 33.27 %. La palabra clave se concatena al texto antes de limpiar, porque señala el tema y aporta un término útil; la ubicación se descarta, porque es texto libre escrito por el usuario y contiene desde países hasta bromas.

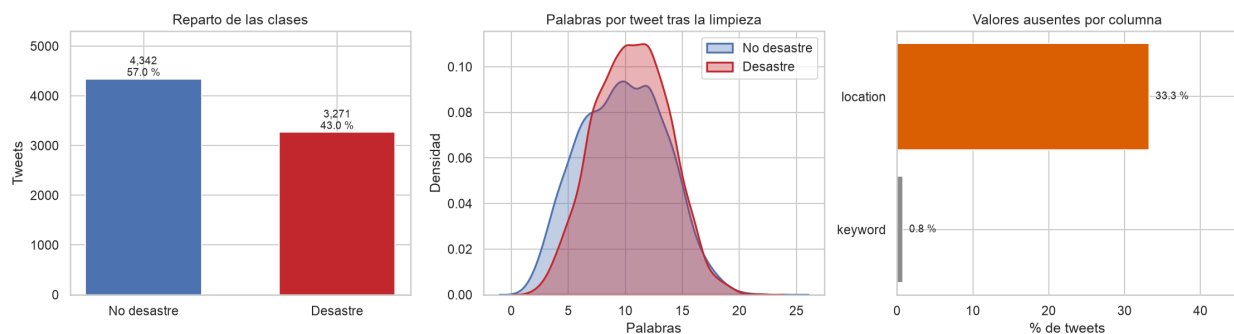


Figura 1. Reparto de clases, longitud de los tweets tras la limpieza y proporción de valores ausentes en las dos columnas auxiliares.

Una observación sobre la calidad del conjunto, detectada al inspeccionar los tweets extremos de la sección 7: **hay duplicados exactos**. El mismo texto aparece repetido carácter por carácter en más de una fila, lo que infla levemente las frecuencias y hace que un mismo tweet pueda caer a la vez en entrenamiento y en validación. No se eliminaron, para no alterar la base sobre la que se hicieron los ejercicios previos, pero conviene tenerlo presente al leer cualquier conteo.

2. Limpieza y preprocesamiento

La limpieza se implementó con **NLTK** (lista de palabras vacías del inglés y lematizador de WordNet), **pandas** y el módulo `re` de la biblioteca estándar. Sobre el texto original se aplican, en este orden:

Paso	Qué hace	Motivo
Minúsculas	<code>text.lower()</code>	Une 'Fire' y 'fire' en un solo término
Quitar URL	<code>http\S+ www.\S+</code>	El enlace acortado no aporta léxico
Quitar no alfabético	<code>[^a-zA-Z\s]</code>	Elimina #, @, apóstrofes, signos y dígitos
Normalizar espacios	<code>\s+ → ' '</code>	Deja un solo espacio entre palabras
Quitar emoji	Rangos Unicode	Sin efecto: el paso anterior ya los borró
Quitar palabras vacías	stopwords de NLTK	Artículos, preposiciones y conjunciones

Tabla 1. Los seis pasos de la limpieza, en el orden en que se aplican.

El orden tiene una consecuencia que conviene documentar: como el filtro de caracteres no alfabéticos corre **antes** que el de emoji, para cuando este último se ejecuta ya no queda ningún símbolo que borrar. El paso es inocuo, no erróneo, y se conservó tal cual para que el texto resultante siga siendo idéntico al que entrenó los modelos.

Hay además un detalle de reproducibilidad que afecta a cualquiera que reejecute el laboratorio. El lematizador de WordNet está envuelto en un bloque que cae a la identidad si el corpus no está descargado, y

en el entorno donde se ejecutó la limpieza esa fue la rama que se tomó: el corpus guardado en `processed/` conserva *deeds* y *us* sin reducir a su lema. Comprobado sobre las 7,613 filas, la limpieza sin lematizar reproduce el **99.92 %** del corpus guardado y con lematización sólo el 40.18 %. El módulo `src/texto.py` replica el comportamiento efectivo y deja la lematización como una opción explícita, para que el texto de un tweet nuevo se limpie igual que el de entrenamiento.

El caso del 911 y de los emoticones

El enunciado pide valorar dos decisiones concretas. Sobre el **911**: la limpieza elimina todos los dígitos, y 911 aparece en **4 tweets** de 7,613. Tres de esos cuatro son desastre real, una proporción del 75 % frente al 42.97 % de base, pero con cuatro observaciones no hay evidencia de nada. Quitarlo no costó ninguna capacidad de clasificación medible.

Sobre los **emoticones**, la respuesta requirió medir. El conjunto **no contiene un solo emoji Unicode** (0 apariciones), de modo que no hay nada que conservar por ese lado; emoticones ASCII sí aparecen, pero en 112 tweets, el 1.47 %. Sería un error, sin embargo, concluir que la limpieza fue inocua para el análisis de sentimiento: lo que destruye no son los emoticones sino señales bastante más frecuentes.

Señal borrada por la limpieza	Tweets afectados	% del conjunto
Emoji Unicode	0	0.00 %
Emoticones ASCII	112	1.47 %
Signos de admiración	719	9.44 %
Negaciones (no, not, never...)	918	12.06 %
Palabras en MAYÚSCULAS	1,487	19.53 %

Tabla 2. Señales presentes en el texto original que la limpieza elimina. La negación es la pérdida grave para el análisis de polaridad.

La negación es la más seria de las tres. Un analizador de polaridad que reciba *not good* sin el *not* —y *not* es palabra vacía, de modo que la limpieza la elimina— leerá *good* y puntuará al revés. Por eso la sección 6 trabaja sobre una segunda limpieza, mucho más suave, reservada al sentimiento.

Con el texto limpio se contaron las palabras de cada categoría por separado. El contraste es inmediato y es la base de que la clasificación funcione: el vocabulario de los tweets de desastre es informativo y concreto —*fire, news, disaster, police, suicide, storm*— mientras que el de la otra categoría es conversacional —*like, im, new, get, dont, love*.

Tabla 3. Las diez palabras más frecuentes en cada categoría.

Palabras más frecuentes — desastre real

Palabra	Apariciones (aprox.)
fire	190
news	140
disaster	135
police	125
via	120
california	110
suicide	110
amp	105
people	105
storm	100
wildfire	95
crash	95

Palabras más frecuentes — no desastre

Palabra	Apariciones (aprox.)
like	250
im	245
amp	195
new	170
get	165
dont	155
one	130
body	115
emergency	105
via	100
fire	100
would	100

[illegible]

Figura 3. Nubes de palabras de las dos categorías. El tamaño es proporcional a la frecuencia.

Bigramas y trigramas

Los n-gramas resuelven parte de la ambigüedad anterior, y por eso vale la pena explorarlos. *Northern california* sólo indica un lugar, pero *northern california wildfire* ya identifica un suceso. Entre los tweets de desastre los bigramas más frecuentes son *suicide bomber* (59), *northern california* (41), *oil spill* (38) y *burning buildings* (35); entre los otros, *cross body* (38), *youtube video* (36) y *liked youtube* (35), que no describen ningún acontecimiento.

Los trigramas afinan más —*suicide bomber detonated*, *northern california wildfire*, *obama declares disaster*— pero su frecuencia cae a menos de treinta apariciones, de modo que aportan precisión a costa de cobertura. Por eso el vectorizador de la sección siguiente se queda en unigramas y bigramas: el trigrama es demasiado escaso para generalizar sobre 7,613 tweets.

4. Modelos de clasificación

La representación es **TF-IDF sobre unigramas y bigramas**, con `min_df=2` para descartar los términos que aparecen una sola vez, `sublinear_tf=True` para amortiguar las repeticiones y un máximo de 100,000 términos. Ésa es la forma en que se aborda el contexto: el bigrama permite que *suicide bomber* pese como una unidad, distinta de la suma de *suicide* y *bomber* por separado.

Se compararon tres algoritmos sobre una partición 80/20 estratificada con semilla 42: Naive Bayes multinomial ($\alpha = 0.5$), regresión logística ($C = 2.0$, hasta 1,000 iteraciones) y SVM lineal ($C = 1.0$). Los tres reciben exactamente la misma matriz de características.

Modelo	Exactitud	Precisión	Recall	F1
Naive Bayes	0.8234	0.8612	0.7018	0.7734
Regresión Logística	0.8162	0.8224	0.7294	0.7731
SVM lineal	0.7978	0.7772	0.7416	0.7590

Tabla 4. Métricas sobre la partición fija 80/20 (1,523 tweets de validación).

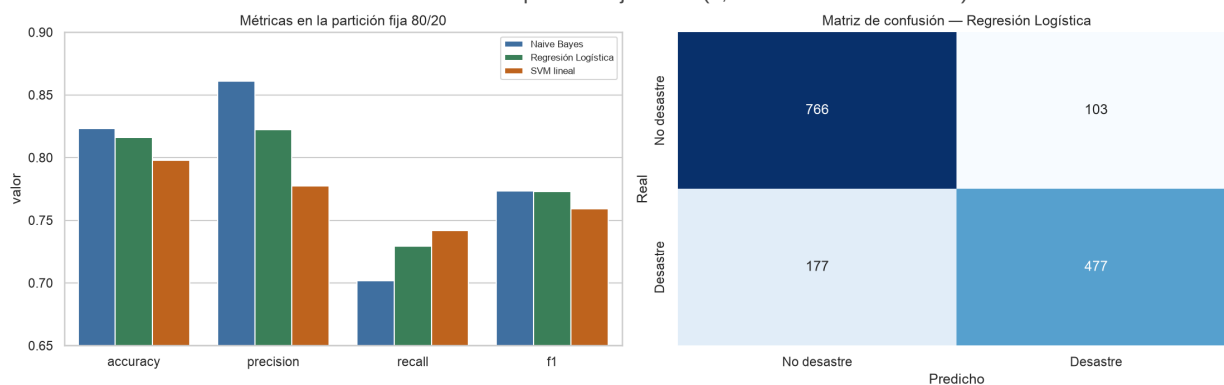


Figura 4. Comparación de métricas y matriz de confusión del modelo seleccionado (Regresión Logística).

Por qué el mejor modelo no es el que parecía

Sobre esa partición gana **Naive Bayes** con $F1 = 0.7734$. El problema es que la diferencia con el segundo es de tres diezmilésimas, y una diferencia así sobre 1,523 tweets no distingue un modelo mejor de una partición afortunada. Se repitió la comparación con validación cruzada estratificada de 5 pliegues y 3 repeticiones, quince ajustes por modelo:

Modelo	F1 partición fija	F1 medio (15 pliegues)	Desviación
Regresión Logística	0.7731	0.7581	± 0.0093
Naive Bayes	0.7734	0.7445	± 0.0133
SVM lineal	0.7590	0.7442	± 0.0102

Tabla 5. La partición fija y la validación cruzada no coinciden en el orden de los modelos.

El orden se invierte. **Regresión Logística** alcanza F1 medio de 0.7581 frente a 0.7445 de Naive Bayes, y la ventaja de 1.4 puntos supera con holgura las desviaciones de ambos. Se selecciona por tanto la **regresión logística**. La lectura general es que con conjuntos de este tamaño la selección de modelo no debería descansar en una sola partición.

Sobre las métricas: el modelo es más preciso que sensible. Recupera alrededor del 73 % de los tweets de desastre y acierta en cerca del 82 % de los que marca. Para un sistema de alerta el error grave sería el falso negativo —un desastre que pasa inadvertido— de modo que un despliegue real bajaría el umbral de decisión para ganar recall a costa de precisión.

5. Función de clasificación

La función `clasificar_tweet(tweet)` recibe el texto **sin preprocesar**, le aplica exactamente la misma limpieza que se usó en el entrenamiento y devuelve la clase junto con la probabilidad estimada. El modelo se reentrena antes con las 7,613 filas, no sólo con el 80 %, porque una vez elegido el algoritmo no hay razón para desperdiciar datos.

Tweet de entrada	Salida	p(desastre)
Massive earthquake destroys buildings and rescue teams...	desastre real	0.9065
I had an amazing day at the beach with my friends	no desastre	0.1008
Forest fire near La Ronge Sask. Canada	desastre real	0.9281
This new album is absolute fire :) I am obsessed	no desastre	0.2223
BREAKING: 13 dead after suicide bomber targets mosque...	desastre real	0.9010

Tabla 6. Comportamiento de la función sobre cinco tweets nuevos.

El cuarto caso es el interesante. *This new album is absolute fire* contiene la palabra que encabeza el vocabulario de desastre, y aun así el modelo le asigna 0.2223 de probabilidad: los bigramas *new album* y *absolute fire* aportan el contexto que *fire* por sí sola no tiene. Es exactamente la ambigüedad que se anticipó en la sección 3, resuelta por la representación.

6. Análisis de sentimiento

La herramienta es **VADER** (Hutto y Gilbert, 2014), un léxico con reglas construido y validado sobre texto de redes sociales. Frente a un conteo simple de palabras positivas y negativas, VADER modela la negación, los intensificadores, el énfasis por mayúsculas y signos de admiración, y los emoticones. Devuelve una puntuación compuesta en $[-1, 1]$; el corte estándar de sus autores clasifica como positivo por encima de $+0.05$, negativo por debajo de -0.05 y neutro en medio.

Siguiendo lo discutido en la sección 2, se puntúa sobre una **segunda limpieza** que sólo quita URL, menciones y la almohadilla de los hashtag, conservando mayúsculas, puntuación, negaciones y emoticones. La decisión no es cosmética: entre puntuar el texto del ejercicio 3 y puntuar éste, **el 7.46 % de los tweets cambia de signo**, y la media de la puntuación compuesta pasa de -0.2080 a -0.1441 . La limpieza agresiva hace que el corpus parezca más negativo de lo que es, porque al borrar las negaciones *not bad* se contabiliza como *bad*.

Categoría	Negativo	Neutro	Positivo
No desastre	42.81 %	24.34 %	32.84 %
Desastre real	57.72 %	25.80 %	16.48 %

Tabla 7. Reparto de polaridad dentro de cada categoría.

Conviene fijarse en dónde está la diferencia. En el extremo negativo la brecha es de 15 puntos, un factor de 1.35; en el positivo es de 16 puntos pero sobre una base mucho menor, un factor de **2.0**. Lo que distingue a un tweet de desastre no es tanto que sea más negativo como que **casi nunca es positivo**. La proporción de neutros es prácticamente igual en ambas categorías.

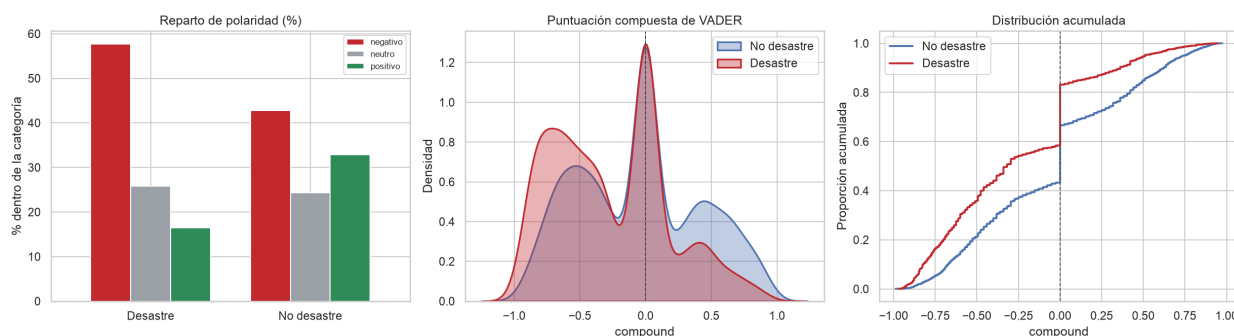


Figura 5. Reparto de polaridad, densidad de la puntuación compuesta y distribución acumulada, por categoría.

Una advertencia sobre qué mide esta variable. Las palabras negativas más frecuentes del corpus son *no*, *fire*, *emergency*, *disaster*, *crash*, *suicide*, *bomb*, *killed*, *dead*, *attack*. Salvo la primera, son el vocabulario de desastre. La negatividad que se está midiendo no es una dimensión independiente del texto sino un promedio de la presencia de palabras que el clasificador ya usa por separado. Es la clave para entender la sección 8.

Como contraste se repitió todo con **TextBlob**. El acuerdo entre ambos léxicos es del 46.81 %, bajo, y el desacuerdo dominante son 1,821 tweets que VADER llama negativos y TextBlob neutros. La causa es que el analizador de TextBlob se apoya en un diccionario de adjetivos, y *killed*, *bomb* o *collapse* son sustantivos y verbos que no puntúa. Para este corpus VADER es el instrumento adecuado; TextBlob se conserva para verificar direcciones, no para clasificar.

7. Los tweets extremos y la comparación entre categorías

Los 10 más negativos y los 10 más positivos

Siete de los diez tweets más negativos son desastre real, contra una prevalencia de base del 42.97 %: atentados suicidas, bombardeos de mezquitas y asesinatos. Los otros tres muestran el modo de fallo del método. El tweet más negativo de todo el conjunto **no es un desastre**: es *wreck? wreck wreck wreck...*, la misma palabra repetida trece veces, que basta para llevar la puntuación a -0.9883 porque VADER suma la valencia de cada aparición. Los otros dos son un tweet de fandom y un pez muerto en un lago.

Nueve de los diez más positivos no son desastre. El único de la categoría de desastre usa la metáfora *today's storm will pass*, donde *storm* no se refiere a ninguna tormenta; es discutible que la etiqueta sea correcta y es la clase de caso que ningún método léxico resuelve. La lección de conjunto es que la polaridad mide **intensidad léxica acumulada, no acontecimientos**.

Veinte tweets son pocos para afirmar un patrón, así que se amplió la ventana. Entre los 500 más negativos el 72.8 % son desastre y entre los 500 más positivos, el 18.6 %, frente al 42.97 % de base. El patrón se sostiene y no es un artefacto de haber mirado sólo diez.

¿Son los tweets de desastre más negativos?

Sí, y la afirmación resiste el contraste, pero el efecto es moderado. Se usó la U de Mann-Whitney y no la t de Student porque la puntuación compuesta está acotada, tiene una masa puntual en el cero —una cuarta parte de los tweets es neutra— y no es normal.

Medida	Desastre real	No desastre
Tweets	3,271	4,342
Media de compound	-0.2663	-0.0520
Mediana	-0.3182	0.0000

Tabla 8. Polaridad por categoría según VADER.

La diferencia de medias es de -0.2143, con un intervalo de confianza al 95 % por bootstrap de [-0.2335, -0.1927] que no se acerca al cero, y $p = 2.7e-85$. Ese valor p , sin embargo, dice poco: con 7,613 observaciones casi cualquier diferencia sale significativa. **La cifra que importa es el tamaño del efecto**. La probabilidad de que un tweet de desastre tomado al azar sea más negativo que uno de la otra categoría es **0.6297**, y la correlación biserial por rangos +0.2593, un efecto pequeño a moderado. En más de un tercio de las parejas que se comparen, el tweet de desastre será el menos negativo de los dos.

El mismo contraste con TextBlob da un efecto de 0.1115, menos de la mitad, pero de la misma dirección y con la misma significancia. Que el efecto se atenúe al cambiar de instrumento pero **nunca se invierta** es la mejor evidencia de que la diferencia entre categorías es real.

8. La variable de negatividad y el reentrenamiento

Se creó la variable negatividad como la puntuación compuesta invertida y reescalada a $[0, 1]$, donde 0 es el tweet más positivo posible y 1 el más negativo. Se dejó sin valores negativos a propósito, porque Naive Bayes multinomial no admite entradas negativas. La variable forma parte del conjunto guardado en `processed/train_sentimiento.csv`.

Antes de añadirla se midió cuánto vale sola: una regresión logística que no ve el texto y sólo usa la negatividad alcanza **exactitud de 0.60**, frente al 0.5706 de responder siempre «no desastre». Tres puntos por encima de no hacer nada, lo que ya anticipaba el resultado.

Modelo	F1 sin	F1 con	Δ F1 (fija)	Δ F1 (15 pliegues)
Naive Bayes	0.7734	0.7734	+0.0000	+0.0001
Regresión Logística	0.7731	0.7756	+0.0025	-0.0014
SVM lineal	0.7590	0.7600	+0.0010	+0.0032

Tabla 9. Efecto de añadir la negatividad, en la partición fija y promediado sobre quince pliegues.

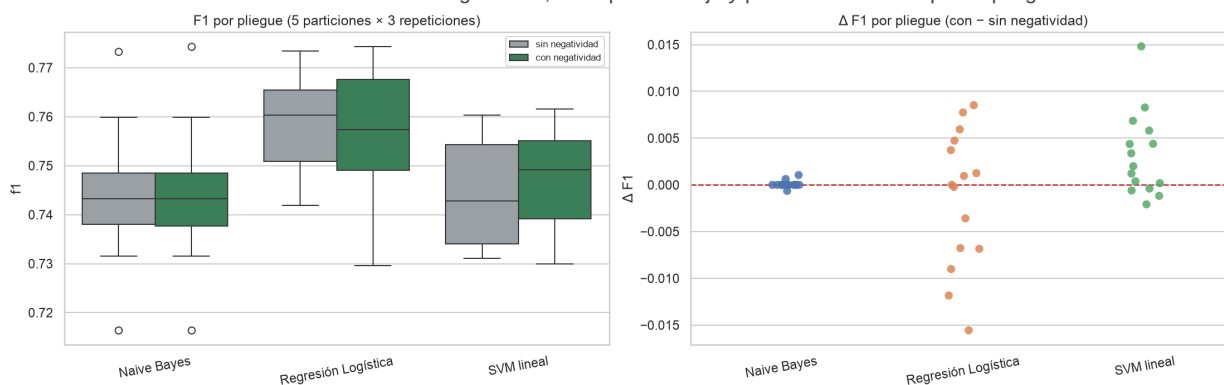


Figura 6. F1 por pliegue en las dos condiciones y diferencia pliegue a pliegue. La línea roja marca el cero.

Respuesta: ¿mejoró, y en qué medida?

No de manera que importe. En la partición fija las diferencias van de 0.0000 a +0.0025 de F1; sobre quince pliegues, de -0.0014 a +0.0032, y sólo el SVM lineal alcanza significancia ($p = 0.016$) con una mejora de tres milésimas que además lo deja por debajo de donde ya estaba la regresión logística sin la variable.

La prueba más directa es la de McNemar sobre las predicciones del mejor modelo. Sobre los mismos 1,523 tweets de validación, añadir la negatividad **arregla 30 clasificaciones y rompe 26**; en 1,467 los dos modelos coinciden. Con $p = 0.6889$, esa diferencia es indistinguible de lanzar una moneda 56 veces. Si hay que dar una cifra, la mejora es de **+0.0025 de F1**, unos cuatro tweets de 1,523, y no sobrevive al cambio de partición.

Tampoco ayuda entregar el bloque completo de seis puntuaciones de sentimiento: la regresión logística gana siete milésimas y Naive Bayes **empeora** seis, que es lo esperable de un modelo que supone independencia entre características cuando recibe seis variables fuertemente correlacionadas.

Por qué no mejora. La explicación está en la sección 6. Las palabras que aportan negatividad en este corpus son el vocabulario de desastre. El TF-IDF ya usa cada una como predictora independiente, con un peso aprendido de estos datos y de esta tarea; la variable de negatividad toma esas mismas palabras, las promedia con pesos fijos traídos de un léxico calibrado para medir tono, y comprime el resultado en un solo número. No aporta información nueva sino una versión comprimida y con pérdida de información que el modelo ya tenía en mejor forma. Cabría esperar que sí aportara si el clasificador no viera el texto, o si la tarea fuera distinguir el tono en lugar del

tema.

9. Conclusiones y limitaciones

El problema se resuelve razonablemente bien con métodos léxicos clásicos: F1 de 0.7581 con TF-IDF de unigramas y bigramas y regresión logística. El contexto entra por la vía del bigrama, y basta para resolver casos como *absolute fire*, donde la palabra aislada apuntaría a la clase contraria.

El análisis de sentimiento confirma que los tweets de desastre son más negativos, con un efecto moderado y consistente entre dos léxicos independientes. Pero esa negatividad resultó ser un reflejo del mismo vocabulario que el clasificador ya explota, y por eso incorporarla como variable no mejora nada. El resultado negativo es informativo: señala que la polaridad y el tema están confundidos en este corpus.

Limitaciones

Duplicados. El conjunto contiene tweets repetidos carácter por carácter, que pueden caer a la vez en entrenamiento y validación e inflar levemente las métricas. No se eliminaron para no alterar la base de los ejercicios previos.

Etiquetas discutibles. Varios tweets del análisis de extremos están etiquetados de forma cuestionable, como el uso metafórico de *storm*. Marca un techo al desempeño alcanzable que ningún modelo puede superar.

Léxico ajeno al dominio. VADER está calibrado sobre redes sociales generales, no sobre cobertura de desastres. Un léxico específico del dominio podría separar mejor el tono del tema.

Sin modelos contextuales. No se probaron representaciones basadas en transformadores, que resolverían la ambigüedad léxica mejor que el bigrama, a costa de interpretabilidad y de coste computacional.

Referencias

- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. Módulo NLTK, versión 3.10.3.
- Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*. Paquete vaderSentiment 3.3.2.
- Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3.^a ed., borrador). Capítulos sobre n-gramas y clasificación de textos. <https://web.stanford.edu/~jurafsky/slp3/>
- Kaggle. *Natural Language Processing with Disaster Tweets*. <https://www.kaggle.com/competitions/nlp-getting-started>
- Loria, S. (2020). *TextBlob: Simplified Text Processing*, versión 0.20.1.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. Versión 1.9.0.